

Competitive Programming

Section 45 Editorial: Flows and Matchings

Competitive Programming Course

2026-07-12

Table of contents

1	Editorial overview	3
1.1	Problem-specific proof map	3
1.2	Detailed solution guide	3
2	A. Max Flow	5
2.1	Restatement	5
2.2	Observations and algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	5
2.7	Tests that matter	6
3	B. Bipartite Matching	7
3.1	Restatement	7
3.2	Observations and algorithm	7
3.3	Correctness proof	7
3.4	Complexity	7
3.5	Full C++ solution	7
3.6	Full Python solution	7
3.7	Tests that matter	8
4	C. Min Cut Reachable Side	9
4.1	Restatement	9
4.2	Observations and algorithm	9
4.3	Correctness proof	9
4.4	Complexity	9
4.5	Full C++ solution	9
4.6	Full Python solution	9
4.7	Tests that matter	10
5	D. Project Selection	11
5.1	Restatement and reduction target	11
5.2	Network construction and trace	11
5.3	Algorithm	11
5.4	Correctness proof	12
5.5	Complexity, implementation notes, and tests	12
5.6	Full C++ solution	12
5.7	Full Python solution	14
6	What to carry forward	17
7	Testing discussion	18

1 Editorial overview

1.1 Problem-specific proof map

A. Max Flow. Dinic augments only along residual paths, preserving capacity and flow conservation. When no residual s -to- t path remains, reachable residual vertices define a cut saturated by the flow, proving optimality.

B. Bipartite Matching. Unit-capacity source-left, compatibility, and right-sink edges make integral flow paths correspond one-for-one with matched pairs. Thus the maximum flow value is the maximum matching size.

C. Min Cut Reachable Side. After maximum flow, a residual BFS reaches exactly the source side of the minimum cut: any forward edge leaving it has zero residual capacity, while all reachable edges stay on that side.

D. Project Selection. A source-side project is selected. Positive project values become source edges, negative values become sink edges, and each prerequisite becomes an infinite-capacity implication edge. The lost positive value plus selected negative cost is precisely a cut capacity.

1.2 Detailed solution guide

A. Maximum Flow. Dinic's algorithm works on the residual graph. A BFS builds levels using only edges with positive residual capacity. A DFS then sends flow only from level d to level $d+1$, which forms a blocking flow for the current level graph. When the sink is unreachable in BFS, there is no augmenting path left. By the max-flow min-cut theorem, the current flow is maximum. The reverse edge stores the option to cancel earlier choices, which is essential for correctness.

B. Bipartite Matching. Model each left vertex as choosing at most one right vertex. The DFS tries to match a left vertex directly, and if the right vertex is already used, it recursively asks the previous owner to move elsewhere. A successful recursive move creates an augmenting path and increases the matching size by one. When no DFS from any unmatched left vertex succeeds, no augmenting path remains, so the matching is maximum.

C. Minimum Cut After Flow. After computing max flow, run a reachability search from the source in the final residual graph. Let S be the reachable set. No residual edge of positive capacity crosses from S to $V \setminus S$; otherwise the endpoint would be reachable. Thus all original capacity across that cut is saturated, and its value equals the max flow. The printed cut is therefore a minimum cut.

Common pitfalls. Always update reverse edges together with forward edges. For matching, reset the visited array for each DFS root. For min cut, use the residual graph after max flow, not the original graph.

The three exercises are different views of augmenting paths.

- A implements Dinic. BFS builds a residual level graph; DFS sends blocking flow along increasing levels; reverse edges allow rerouting earlier choices.
- B solves bipartite matching with augmenting DFS. A right vertex can be reassigned if its current left partner can find another route.
- C runs max flow, then explores the residual graph from the source. The reachable set is the source side of a minimum cut.

For max-flow correctness, the key terminal condition is: if the sink is not reachable in the residual graph, there is no augmenting path, so the current flow is maximum.

2 A. Max Flow

2.1 Restatement

Given a directed capacitated graph from node 1 to node n , print the maximum flow.

2.2 Observations and algorithm

Dinic works on residual edges. BFS assigns shortest residual levels; DFS pushes only along positive edges from level d to $d+1$ until that level graph blocks all source-sink paths. Update the reverse residual edge whenever flow is pushed.

2.3 Correctness proof

Residual capacities preserve capacity and conservation. When BFS cannot reach sink, no augmenting path remains; max-flow/min-cut proves the flow maximum. QED.

2.4 Complexity

Dinic is $O(V^2 E)$ in the general integral-capacity bound, with $O(V+E)$ graph storage.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>>g;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Dinic g(n);
```

2.6 Full Python solution

```
import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
```

```

flow=0
while True:
    level=[-1]*n; level[s]=0; dq=deque([s])
    while dq:
        u=dq.popleft()
        for v,c,rev in g[u]:
            if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
    if level[t]<0: break
    it=[0]*n
    def dfs(u,f):
        if u==t: return f
        for i in range(it[u],len(g[u])):
            it[u]=i; v,c,rev=g[u][i]
            if c and level[v]==level[u]+1:
                ret=dfs(v,min(f,c))
                if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
        return 0
    while True:
        pushed=dfs(s,10**18)
        if not pushed: break
        flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    print(dinic(n,edges,0,n-1)[0])
if __name__=="__main__": main()

```

2.7 Tests that matter

Test no path, parallel edges, a bottleneck, and a rerouting-required network.

3 B. Bipartite Matching

3.1 Restatement

Given edges between left and right parts, print the maximum matching size.

3.2 Observations and algorithm

Use augmenting DFS. A right vertex is free, or its current left owner must first move through an alternate edge. The successful recursion chain is one augmenting path. For L1-R1,R2 and L2-R1, reroute L1 to R2 then match L2-R1.

3.3 Correctness proof

Each success increases matching size by one. If every unmatched left root fails, there is no augmenting path, so the matching is maximum. QED.

3.4 Complexity

The Kuhn DFS implementation is $O(L \cdot E)$ worst case and uses $O(L+R+E)$ memory.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m,e;if(!
```

3.6 Full Python solution

```
import sys, collections
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m,e=data[0],data[1],data[2]; adj=[[] for _ in range(n)]; idx=3
    for _ in range(e): adj[data[idx]-1].append(data[idx+1]-1); idx+=2
    match=[-1]*m
    def dfs(u,vis):
```

```

    for v in adj[u]:
        if vis[v]: continue
        vis[v]=1
        if match[v]==-1 or dfs(match[v],vis): match[v]=u; return True
    return False
ans=0
for u in range(n): ans+=dfs(u,[0]*m)
print(ans)
if __name__=="__main__": main()

```

3.7 Tests that matter

Reset visited-right per root. Test no edges, a reroute case, duplicates, and unequal part sizes.

4 C. Min Cut Reachable Side

4.1 Restatement

After max flow from 1 to n, print the flow value and the source-side reachable vertices in the residual graph.

4.2 Observations and algorithm

After max flow, BFS from source through positive final residual edges. Print the reachable vertices as S together with the flow value.

4.3 Correctness proof

No residual forward edge leaves S, so every original edge from S is saturated. That cut has capacity equal to the maximum flow and is a minimum cut. QED.

4.4 Complexity

The extra residual BFS is $O(V+E)$ after the max-flow computation.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std; struct Dinic{struct E{int v,rev; long long c;};int n;vector<vector<E>>>g;
int main(){ios::sync_with_stdio(false);cin.tie(nullptr);int n,m;if(!(cin>>n>>m)) return 0;Dinic g(n);
```

4.6 Full Python solution

```
import sys
from collections import deque
def dinic(n,edges,s,t,cut=False):
    g=[[] for _ in range(n)]
    def add(u,v,c):
        g[u].append([v,c,len(g[v])]); g[v].append([u,0,len(g[u])-1])
    for u,v,c in edges: add(u,v,c)
```

```

flow=0
while True:
    level=[-1]*n; level[s]=0; dq=deque([s])
    while dq:
        u=dq.popleft()
        for v,c,rev in g[u]:
            if c and level[v]<0: level[v]=level[u]+1; dq.append(v)
    if level[t]<0: break
    it=[0]*n
    def dfs(u,f):
        if u==t: return f
        for i in range(it[u],len(g[u])):
            it[u]=i; v,c,rev=g[u][i]
            if c and level[v]==level[u]+1:
                ret=dfs(v,min(f,c))
                if ret: g[u][i][1]-=ret; g[v][rev][1]+=ret; return ret
        return 0
    while True:
        pushed=dfs(s,10**18)
        if not pushed: break
        flow+=pushed
    if cut:
        seen=[False]*n; dq=deque([s]); seen[s]=True
        while dq:
            u=dq.popleft()
            for v,c,rev in g[u]:
                if c and not seen[v]: seen[v]=True; dq.append(v)
        return flow,seen
    return flow,None
def main():
    data=list(map(int,sys.stdin.buffer.read().split()))
    if not data: return
    n,m=data[0],data[1]; idx=2; edges=[]
    for _ in range(m): edges.append((data[idx]-1,data[idx+1]-1,data[idx+2])); idx+=3
    flow,seen=dinic(n,edges,0,n-1,True); print(flow); print(' '.join(str(i+1) for i,x in enumerate(seen) if x))
if __name__=="__main__": main()

```

4.7 Tests that matter

Test source-only reachability, zero flow, and parallel cut edges. Use the final residual graph, never the original graph.

5 D. Project Selection

5.1 Restatement and reduction target

Each project has a possibly negative profit. A directed rule $a \rightarrow b$ means that selecting project a requires selecting project b . Choose any feasible set of projects, including the empty set, with maximum total profit. There can be up to 200 projects and 1,000 rules, so enumerating all subsets is impossible.

This is a maximum-closure problem. A cut represents which projects are chosen: put a project on the source side exactly when it is selected. We will construct the network so that a finite cut is exactly a feasible choice, and its capacity is the profit lost by that choice.

5.2 Network construction and trace

Let P be the sum of all positive project profits. Build a source s and sink t around the project vertices:

```
profit w > 0:  add s -> project with capacity w
profit w < 0:  add project -> t with capacity -w
rule a requires b: add a -> b with capacity INF
```

Choose $INF = P + 1$. No optimum cut can afford an INF edge because cutting every positive source edge costs at most P . Therefore a finite minimum cut never puts a on the source side while b is on the sink side: it obeys every requirement.

For profits $[9, -5, 7, -3]$ with rules $1 \rightarrow 2$, $3 \rightarrow 4$, and $3 \rightarrow 2$, selecting all projects earns $9 - 5 + 7 - 3 = 8$. In the corresponding cut, no positive source edge is cut and the two negative project-to-sink edges cost $5 + 3 = 8$. Since $P = 16$, $P - \text{cut} = 8$. Selecting only project 1 and its prerequisite 2 has profit 4 and cuts source edge for project 3 (cost 7) plus project 2's sink edge (cost 5), again $16 - 12 = 4$.

5.3 Algorithm

```
P = sum of positive profits
INF = P + 1
build the stated flow network
min_cut = maximum_flow(s, t)
print P - min_cut
```

The max-flow/min-cut theorem supplies the minimum cut value. Use `long long` for P , INF , capacities, and the result.

5.4 Correctness proof

Consider any feasible selected set S and place exactly its projects with the source. No prerequisite edge crosses from source to sink because feasibility ensures every requirement of a selected project is also selected. The cut crosses $s \rightarrow v$ for every unselected positive project, and $v \rightarrow t$ for every selected negative project. Its capacity is therefore

```
sum(positive profits of unselected projects)
+ sum(absolute negative profits of selected projects)
= P - profit(S).
```

Conversely, a minimum cut cannot cross an INF edge: cutting all source edges has cost $P < \text{INF}$. Thus its source-side projects satisfy every prerequisite, so they form a feasible set, and the same equality shows their profit is $P - \text{cut_capacity}$. Minimizing cut capacity therefore maximizes feasible profit. By max-flow/min-cut, the computed maximum flow is that minimum capacity, so the algorithm prints the optimal profit. QED.

5.5 Complexity, implementation notes, and tests

The network has $n+2$ vertices and $n+m$ forward edges. It uses $O(n+m)$ memory. With arbitrary integral capacities, Dinic has worst-case $O(V^2 E)$ time; the deliberately moderate local bound ($n \leq 200, m \leq 1000$) makes this implementation appropriate in both supplied languages. For substantially larger arbitrary-capacity maximum-closure networks, use a flow implementation and limits whose performance guarantee has been analyzed for that setting. INF must be strictly larger than the total positive profit, not a small magic constant. Test all-negative projects (answer zero), a profitable project whose requirements make it unprofitable, independent profitable projects, and rules forming a cycle.

5.6 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Dinic {
    struct Edge {
        int to, reverse;
        long long capacity;
    };

    vector<vector<Edge>> graph;
    vector<int> level, next_edge;

    explicit Dinic(int n) : graph(n), level(n), next_edge(n) {}

    void add_edge(int from, int to, long long capacity) {
        Edge forward{to, (int)graph[to].size(), capacity};
        Edge backward{from, (int)graph[from].size(), 0};
```

```

        graph[from].push_back(forward);
        graph[to].push_back(backward);
    }

    bool build_levels(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        level[source] = 0;
        queue.push(source);
        while (!queue.empty()) {
            int vertex = queue.front();
            queue.pop();
            for (const Edge& edge : graph[vertex]) {
                if (edge.capacity > 0 && level[edge.to] == -1) {
                    level[edge.to] = level[vertex] + 1;
                    queue.push(edge.to);
                }
            }
        }
        return level[sink] != -1;
    }

    long long send_flow(int vertex, int sink, long long pushed) {
        if (vertex == sink) return pushed;
        for (int& edge_index = next_edge[vertex];
             edge_index < (int)graph[vertex].size(); edge_index++) {
            Edge& edge = graph[vertex][edge_index];
            if (edge.capacity == 0 || level[edge.to] != level[vertex] + 1) continue;
            long long flow = send_flow(edge.to, sink, min(pushed, edge.capacity));
            if (flow == 0) continue;
            edge.capacity -= flow;
            graph[edge.to][edge.reverse].capacity += flow;
            return flow;
        }
        return 0;
    }

    long long max_flow(int source, int sink) {
        long long total = 0;
        const long long INF = numeric_limits<long long>::max() / 4;
        while (build_levels(source, sink)) {
            fill(next_edge.begin(), next_edge.end(), 0);
            while (long long pushed = send_flow(source, sink, INF)) total += pushed;
        }
        return total;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

```

```

int n, m;
if (!(cin >> n >> m)) return 0;
vector<long long> profit(n);
long long positive_total = 0;
for (long long& value : profit) {
    cin >> value;
    if (value > 0) positive_total += value;
}

int source = n;
int sink = n + 1;
Dinic flow(n + 2);
long long inf = positive_total + 1;
for (int project = 0; project < n; project++) {
    if (profit[project] > 0) flow.add_edge(source, project, profit[project]);
    if (profit[project] < 0) flow.add_edge(project, sink, -profit[project]);
}
for (int i = 0; i < m; i++) {
    int project, prerequisite;
    cin >> project >> prerequisite;
    flow.add_edge(project - 1, prerequisite - 1, inf);
}

cout << positive_total - flow.max_flow(source, sink) << '\n';
return 0;
}

```

5.7 Full Python solution

```

import sys
from collections import deque

class Dinic:
    def __init__(self, size: int) -> None:
        self.graph: list[list[list[int]]] = [[] for _ in range(size)]
        self.level = [-1] * size
        self.next_edge = [0] * size

    def add_edge(self, start: int, end: int, capacity: int) -> None:
        self.graph[start].append([end, len(self.graph[end]), capacity])
        self.graph[end].append([start, len(self.graph[start]) - 1, 0])

    def build_levels(self, source: int, sink: int) -> bool:
        self.level = [-1] * len(self.graph)
        self.level[source] = 0
        queue = deque([source])
        while queue:

```

```

        vertex = queue.popleft()
        for target, _, capacity in self.graph[vertex]:
            if capacity > 0 and self.level[target] == -1:
                self.level[target] = self.level[vertex] + 1
                queue.append(target)
        return self.level[sink] != -1

def send_flow(self, vertex: int, sink: int, pushed: int) -> int:
    if vertex == sink:
        return pushed
    while self.next_edge[vertex] < len(self.graph[vertex]):
        edge = self.graph[vertex][self.next_edge[vertex]]
        target, reverse, capacity = edge
        if capacity > 0 and self.level[target] == self.level[vertex] + 1:
            flow = self.send_flow(target, sink, min(pushed, capacity))
            if flow:
                edge[2] -= flow
                self.graph[target][reverse][2] += flow
                return flow
        self.next_edge[vertex] += 1
    return 0

def max_flow(self, source: int, sink: int) -> int:
    total = 0
    while self.build_levels(source, sink):
        self.next_edge = [0] * len(self.graph)
        while pushed := self.send_flow(source, sink, 10**30):
            total += pushed
    return total

def main() -> None:
    tokens = list(map(int, sys.stdin.buffer.read().split()))
    if not tokens:
        return
    n, m = tokens[0], tokens[1]
    profit = tokens[2 : 2 + n]
    pointer = 2 + n
    positive_total = sum(value for value in profit if value > 0)
    source, sink = n, n + 1
    flow = Dinic(n + 2)
    inf = positive_total + 1
    for project, value in enumerate(profit):
        if value > 0:
            flow.add_edge(source, project, value)
        elif value < 0:
            flow.add_edge(project, sink, -value)
    for _ in range(m):
        project, prerequisite = tokens[pointer] - 1, tokens[pointer + 1] - 1
        pointer += 2
        flow.add_edge(project, prerequisite, inf)

```

```
    print(positive_total - flow.max_flow(source, sink))

if __name__ == "__main__":
    sys.setrecursionlimit(300_000)
    main()
```

6 What to carry forward

Advanced algorithms become manageable when you state the invariant first, then code only the operations needed to preserve it.

7 Testing discussion

For tiny random networks, compare with a slow augmenting-path implementation; for matching, compare with brute force over small left vertices. Verify cut output by checking every residual-reachable edge and summing original capacities leaving the reached set. For D, enumerate small project subsets and reject ones that violate a prerequisite before comparing their total profit. Run:

```
CP_TARGET=solution python3 sections/45_flows_and_matchings/check.py
```